

Source-Obligation-Guided Validation of LLM-Based Code Translation

Thuan Mai Duc[✉], Hieu Le Minh[✉], Nguyen Nguyen Phung Trung[✉], Xuan-Bach
Le^{✉*}

Faculty of Computer Science and Engineering, Ho Chi Minh University of
Technology, VNU-HCM, Ho Chi Minh City, Vietnam
{thuan.maiduc2411, hieu.leminh1hks, nguyen.nguyen938,
lexuanbach}@hcmut.edu.vn

Abstract. LLM-translated code can compile, look idiomatic, and pass shallow tests while still changing boundary behavior. We present Sage-TransVal, a validation layer that extracts branch, boundary/type, API, and exception obligations from source programs, turns them into validation cases, compares matched source and target executions, and emits structured counterexample packets. We evaluate 100 Python-to-Java subjects (200 subject-provider trials) and a 30-subject C-to-Rust corpus (60 trials), translated by GPT-4o-mini and Gemini 2.5 Flash Lite. On Python-to-Java, obligation-guided validation detects mismatches in 79.0% of trials, compared with 81.0% for the generic boundary-test baseline; the paired comparison finds no statistically significant difference (McNemar $p = 0.42$). Guided suites average 4.92 tests per trial, compared with 5.77 for the baseline (14.7% fewer; Wilcoxon $p = 2.0 \times 10^{-6}$). On C-to-Rust, the observed rates are 11.7% (7/60) for obligation-guided validation and 10.0% (6/60) for the generic baseline. These results are descriptive. Disabling the primary UB-safe policy changes the C-to-Rust detection rate to 41.7%, primarily through A-integer cases; D-string detection remains 33.3%. We report packet availability and contents, but do not evaluate downstream repair effectiveness.

Keywords: LLM code translation · Translation validation · Static analysis · Differential testing · Fuzzing · Program repair

1 Introduction

LLM-based code translation is moving from demonstrations into migration work, including C-to-Rust conversion and repository-level rewriting [2,19,28]. The harder failure is *false acceptance*: a target program may compile, look idiomatic, and pass the tests already in place while subtly changing semantics. Existing tests are often too sparse for this decision, and broad boundary tests rarely expose which source behavior each input was meant to check.

* Corresponding author.

The artifact of this paper can be found at: GitHub artifact.

Problem setting. Unlike a compiler pass, an LLM translator is prompt-conditioned, black-box, and often optimized for an idiomatic target rather than a structurally close one. That flexibility helps in class-level translation, CUDA-to-HIP migration, C-to-Rust migration, and pseudocode-mediated multilingual translation [23,11,6,18]. It also weakens the usual evidence channels. Syntax is not comparable, intermediate representations are unavailable, and compilation can succeed even when behavior has shifted. Recent validated-transpilation and translation-testing work therefore pushes validation toward behavioral evidence rather than compilation or textual similarity alone [25,3,8].

Distributional program analysis also treats LLMs as black-box program samplers, using invariant survival across generated samples as behavioral evidence [20]. Unlike that cross-sample analysis, SageTransVal derives obligations from a designated source program.

We model validation as bounded behavioral agreement between a source program and its translation. For a generated suite T , we compare source and target observations under an explicit normalization policy:

$$\forall x \in T. O_s(P_s, x) \equiv_{\text{norm}} O_t(P_t, x).$$

Normalization canonicalizes representation choices while preserving semantic differences in values and unexpected exceptions. Because this condition is limited to the tested suite, validation also benefits from tests whose connection to source behavior is visible.

SageTransVal uses source-side behavioral obligations to organize validation. From a source program, we extract branch/control-flow, boundary/type, API, and exception obligations, generate tests from those obligations, and compare source and target observations with an explicit cross-language oracle. When observations diverge, it emits an eight-field counterexample packet with the input, both observations, obligation identity, and the normalization policy. The validation protocol requires a repaired target to pass the full suite; this study does not evaluate repair outcomes.

On 200 Python-to-Java (subject, provider) trials, the generic boundary-test baseline detects mismatches in 81.0% of trials and obligation-guided validation in 79.0%; the paired difference is not statistically significant. Guided validation uses 14.7% fewer tests on average and records the source-obligation family associated with each generated test. On 60 C-to-Rust trials, the observed rates are 11.7% for obligation-guided validation and 10.0% for the generic baseline; we report these results descriptively. The evaluation also measures family-level coverage and packet availability, not the effect of packets on repair.

We organize the evaluation around four questions. **RQ1** asks whether source obligations expose mismatches at competitive rates. **RQ2** asks how suite size compares across methods. **RQ3** asks how obligation-family coverage changes under ablation. **RQ4** examines whether mismatches can be represented in structured packets intended for repair and revalidation. The study evaluates packet generation, not repair-agent performance.

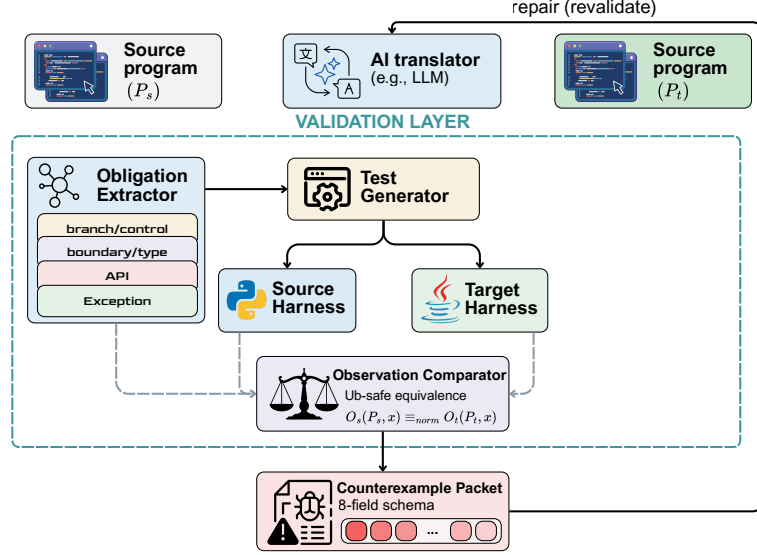


Fig. 1. Validation pipeline for LLM-based code translation. The translator produces a target program, while the validation layer extracts source obligations, generates paired tests, executes source and target harnesses, compares observations under the oracle policy, and records a structured counterexample packet when behavior diverges.

Contributions. We present a translator-agnostic validation architecture, an obligation-family-based test-generation approach, a differential oracle and structured counterexample packet, and an evaluation over Python-to-Java and C-to-Rust that reports mismatch detection, suite size, family coverage, packet availability, and UB-policy sensitivity.

2 Approach

Figure 1 shows the validation path used in the paper. The LLM produces a target program, and SageTransVal derives source obligations, generates paired harness tests, compares source and target observations, and packages detected mismatches in a structured format intended for downstream repair.

Obligations and tests. We use an obligation to mean a testable statement about source behavior. The evaluated extractor reports four families: branch/control-flow obligations (branch outcomes, loop boundaries, guarded paths), boundary/type obligations (empty/null/extreme values, coercions, signedness, integer width, representation changes), API obligations (library preconditions and replacement APIs), and exception obligations (expected errors, panic behavior, propagation). Data-flow obligations remain design scope and are not counted in the reported implementation. Each generated test records its producing obligation family, generator strategy, RNG seed, input, harness, and execution budget, so a passing suite still reports which behavioral families we exercised.

We make the obligation layer source-first. It does not assume that the target preserves statement order, control-flow shape, helper functions, or library names. We derive branch and path obligations from reachable predicates and guard conditions, instantiate boundary/type obligations with empty, null-like, signedness, width, and representation-sensitive values, encode API obligations from documented preconditions and target-language replacement behavior, and capture exception obligations through expected errors, sentinel returns, panic behavior, and propagation paths. Symbolic reasoning supplies high-value seeds when path predicates can be solved, while fuzzing expands around solved seeds under type and harness constraints. The hybrid design follows the testing lesson from symbolic execution and fuzzing work [9,27,16]: precise seeds and broad exploration are complementary, especially when the target is a black-box translation rather than a compiler artifact.

We report obligation-family coverage separately from mismatch detection. For each trial, family coverage is the number of distinct implemented families represented by at least one generated test divided by four; reported values are means across trials. Thus, the reported 40.4% corresponds to an average of about 1.62 represented families per trial, not 40.4% of source code or individual obligations covered. The ablation in Section 4 reports how this measure and mismatch detection change when a family is removed.

We draw the family boundaries pragmatically. Branch and control-flow obligations cover both sides of conditionals, loop entry/exit, and guarded paths that a translator can preserve syntactically but still mishandle through off-by-one or short-circuit changes. Boundary/type obligations cover empty inputs, null-like values, integer extrema, width and signedness changes, representation-sensitive strings, and coercions. API obligations cover library preconditions and replacement APIs, such as a target-language method that is idiomatic but has slightly different error or ordering semantics. Exception obligations cover explicit throws, propagated errors, sentinel-return conversions, and panic behavior. Policy obligations such as C undefined behavior belong in the oracle rather than in a counted implementation family, because they define which observations are comparable.

The family labels group generated tests by the source behavior they are intended to exercise. They can inform later diagnosis or test selection, although this study does not evaluate those downstream uses.

Oracle and packets. Source and target adapters emit a common observation record: return value, output trace, exception category, side effects, and bounded resource outcome. We compare observations after conservative normalization. Numeric formatting is canonicalized, but values are exact unless a declared tolerance applies. Unordered collections are normalized only when source semantics declares them unordered, and time, randomness, filesystem, and network effects are controlled or excluded. Every mismatch yields the canonical eight-field packet:

<code>source_span</code>	<code>target_span</code>
<code>normalized_input</code>	<code>source_observation</code>
<code>target_observation</code>	<code>obligation_id</code>
<code>error_class</code>	<code>normalization_policy</code>

We keep the oracle conservative by construction. Numeric normalization removes superficial formatting differences but does not introduce approximate equality unless a subject declares a tolerance. Exception matching operates over categories rather than language-specific class names, but an unexpected exception is still a mismatch. Collection-order normalization applies only when the source semantics declares order irrelevant. For C-to-Rust validation, C executions that trigger undefined behavior under UBSan are treated as unconstrained under the primary policy: a Rust panic, value, or error on that input is not counted as a mismatch. Section 4 reports a policy-off sensitivity run so this equivalence choice remains visible rather than hidden in the aggregate result.

The packet contains the failing normalized input, both observations, source and target spans when available, the producing obligation, an error class, and the normalization policy used for comparison. Our evaluation measures packet availability when guided validation detects a mismatch; it does not test whether the format improves repair. The validation protocol specifies full-suite revalidation for any repaired target.

Implementation. We implement SageTransVal as a Python research framework with subject loaders, translation caches, language adapters, a differential validator, a packet generator, and report generation. Python functions run through isolated imports. Java targets are compiled and wrapped with `javac/java`, and C and Rust subjects use generated harnesses compiled locally. Runs log subject id, provider, model, validation mode, tests, obligation families, mismatches, first mismatch position, compile/runtime/timeout outcomes, and packet status.

Provider calls are cached by provider, model, prompt, and decoding parameters, so validation is run from saved translations and local execution logs. The artifacts emitted by each run include JSONL events, CSV summaries, LaTeX tables, and PGFPlots figures. The same event log supports the paper’s detection rates, test-count distributions, obligation-family coverage, provider splits, policy sensitivity, and packet availability.

We keep translation, validation, and reporting as separate stages. Translation produces immutable target snapshots. Validation consumes those snapshots and emits event logs without modifying the target. Reporting then derives the paper tables and plots from the logs. A future repair agent can consume the packets, with any repaired target evaluated as a new snapshot under the same validation suite.

3 Evaluation Design

We evaluate mismatch detection, test-suite size, obligation-family coverage, and packet availability. Our primary pair is 100 Python-to-Java benchmark func-

tions from HumanEval/MBPP-style sources [5,1]. Our secondary pair is a 30-subject C-to-Rust corpus targeting three observable seams: A integer (signed-overflow, shift-by-width, INT_MIN division, and defined unsigned-cast cases), C error (sentinel-return paths that idiomatic Rust may recast as `Option/Result`), and D strings (byte-level C string operations on UTF-8 input). We translate each subject with GPT-4o-mini and Gemini 2.5 Flash Lite using temperature 0, a 1200-token translation cap, prompt/model cache keys, and cached responses, yielding 200 Python-to-Java and 60 C-to-Rust trials.

The Python-to-Java subjects are small enough to run repeatedly but diverse enough to include arithmetic, strings, lists, dictionaries, recursion, branch-heavy predicates, and exception-bearing edge cases. We do not use the C-to-Rust subjects to estimate a population-wide migration rate. We use them as a stress set for places where C and Rust disagree about what the validator should count as equivalent. For both pairs, a trial is the tuple of subject, provider, translated target, validation mode, and oracle policy. We record compilation failures, run-time failures, timeouts, and observation mismatches as distinct outcomes before collapsing them into the detection metrics reported in Section 4.

We compare existing seed tests, random/property tests, a generic boundary-test baseline, source-obligation-guided validation, and guided validation with packet emission. For Python-to-Java we also ablate boundary, branch/control-flow, and exception obligations. We report mismatch detection, tests per program, tests per detected mismatch, obligation-family coverage, median test to first mismatch, and packet availability. Tests per detected mismatch is a derived suite-size ratio; we do not use it as a measure of wall-clock or resource cost. C-to-Rust uses `-fsanitize=undefined` on C and `-C overflow-checks=on` on Rust. Under the primary UB-safe policy, C-side undefined behavior is treated as “any” so a Rust panic on a C-undefined input is not itself a mismatch. Table 1 also reports the policy-off sensitivity.

We choose baselines to separate three sources of evidence. Existing tests approximate what a translator would inherit from a benchmark or migration workload. Random/property tests add breadth without source-derived provenance. The template-based generic boundary-test baseline retains the seed tests and adds cases from fixed value pools selected by the argument types in the first seed input (for example, empty strings/lists and small negative, zero, and positive integers). It is deterministic and does not call an LLM or use source-obligation extraction; accordingly, we refer to it as a generic boundary-test baseline, not an LLM-generated baseline. Guided validation with packet emission uses the same tests as obligation-guided validation and records a packet for each detected mismatch; it is reported separately to show packet availability, not as a separate detection method.

For reproducibility, we fix the subject manifests, translation prompts, cached provider responses, validation events, and aggregation scripts. No model is trained or fine-tuned for this study: GPT-4o-mini and Gemini 2.5 Flash Lite are used only as fixed translators, while all test generation, ablation, oracle, and statistical steps are deterministic. A rerun prepares the manifests,

translates each subject once per provider, validates every mode against the same cached target, writes one JSONL event per {subject, provider, mode}, and regenerates the CSV/LaTeX tables and PGFPlots figures from those logs. Each event records dataset, subject id, provider, model, mode, pair, detection flag, mismatch count, first mismatch index, generated-test count, covered obligation families, coverage, packet status, cache mode, and compile/runtime/timeout outcomes before aggregation. We produce the reported statistics from the same event logs with fixed seed 20260516 and 10K bootstrap resamples.

For Python-to-Java, inferential statistics operate on the 200 paired subject-provider trials. Bootstrap intervals summarize detection uncertainty, McNemar exact tests compare paired detection outcomes across modes, and a paired Wilcoxon signed-rank test compares tests/program between the two top raw-detection modes. C-to-Rust is reported descriptively because its 60 trials are split across deliberately different semantic categories and are too small for the same per-method inference.

To keep the comparison paired, every validation mode runs against the same translated target for a given subject and provider. The paired design avoids conflating validator behavior with translator sampling noise. The event log records the first mismatch position and the full generated-test count, so we measure suite size both as total tests/program and as the number of tests required before a mismatch is observed. Tests per detected mismatch is a derived descriptive metric: tests/program divided by detection rate. It does not estimate execution time or overall validation cost; we report it as a normalized suite-size measure.

We fix oracle policies before aggregation. Python-to-Java uses exact value comparison after representation normalization and exception-category mapping. C-to-Rust has two reported policies: the primary UB-safe policy treats C undefined behavior as unconstrained, while the policy-off sensitivity run counts Rust panics on those inputs as mismatches. The two policies therefore answer different equivalence questions; we report both outcomes and identify the primary policy used for the main C-to-Rust results.

4 Results

On Python-to-Java, obligation-guided validation detects mismatches in 79.0% of trials, compared with 81.0% for the generic boundary-test baseline. The paired comparison is not statistically significant. Guided validation uses 4.92 tests per program on average, compared with 5.77 for the baseline. On C-to-Rust, the corresponding observed rates are 11.7% (7/60) and 10.0% (6/60); these results are descriptive. Table 1 also reports how C-to-Rust detection changes under the policy-off oracle.

Table 1. Mismatch detection, suite-size measures, packet availability, and C-to-Rust policy sensitivity.

Pair / view	Mode or category	Trials	Detected	Tests/prog.	Tests/detected	Packet / policy
Python→Java	Existing	200	70.5%	1.00	1.42	—
	Random/property	200	74.5%	3.49	4.68	—
	Generic boundary	200	81.0%	5.77	7.12	—
	Obligation guided	200	79.0%	4.92	6.23	79.0% packet
	Guided+packet	200	79.0%	4.92	6.23	79.0% packet
C→Rust	Existing	60	5.0%	1.00	20.00	—
	Random/property	60	8.3%	3.63	43.60	—
	Generic boundary	60	10.0%	6.23	62.33	—
	Obligation guided	60	11.7%	9.33	80.00	11.7% packet
	Guided+packet	60	11.7%	9.33	80.00	11.7% packet
Policy sensitivity	A integer	22	0.0%	—	—	UB-safe off: 63.6%
	C error	20	5.0%	—	—	UB-safe off: 25.0%
	D strings	18	33.3%	—	—	UB-safe off: 33.3%
	All C/Rust	60	11.7%	—	—	UB-safe off: 41.7%

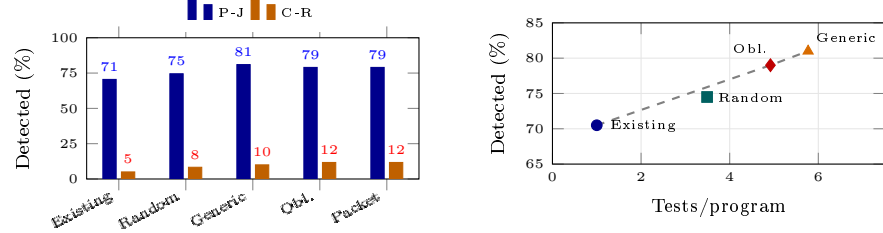
**Fig. 2.** Mismatch detection and suite size. Left: per-pair detection under the primary oracle. Right: Python-to-Java detection rate and mean suite size. Obligation-guided validation has a two-percentage-point lower detection rate and uses 14.7% fewer tests per program than the generic boundary-test baseline. This study does not measure wall-clock savings.

Table 2 and Figure 3 report the ablation results and family coverage. In the displayed C-to-Rust category results, detections under the primary policy occur in the D-string category; A-integer detections appear under the policy-off oracle (Table 1).

Table 2. Obligation-family ablation and C-to-Rust category detection. The ablation preserves the original detection, tests/program, obligation coverage, and median-test evidence, while the category block preserves the C-to-Rust per-category detection table.

Variant	Det.	Tests	Cov.	Med.	Category	Exist.	Rand.	Generic	Obl.	Packet
Full guided	39.5%	5.3	40.4%	1.0	A integer	0.0	0.0	0.0	0.0	0.0
No boundary	39.0%	2.7	23.5%	1.0	C error	0.0	0.0	0.0	5.0	5.0
No branch	39.0%	4.3	31.2%	1.0	D strings	16.7	27.8	33.3	33.3	33.3
No exception	39.5%	5.1	34.4%	1.0	—	—	—	—	—	—

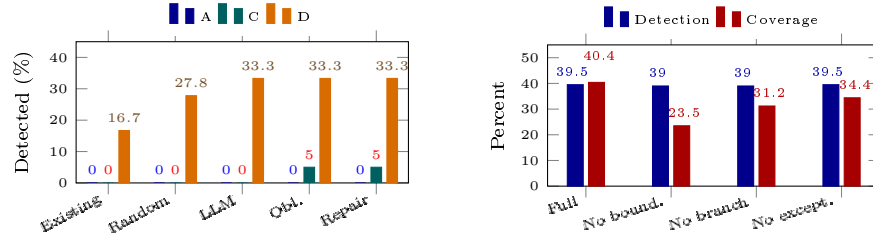


Fig. 3. Category and ablation plots. Left: C-to-Rust primary-policy detection by semantic category. Right: Python-to-Java obligation-family ablation. Coverage falls even when raw detection stays flat, showing why provenance is the main contribution.

The paired statistical tests show that obligation guidance detects more mismatches than existing and random tests in this sample, while the difference from the generic boundary-test baseline is not statistically significant. The paired Wilcoxon test indicates a lower test count per program for obligation-guided validation. Provider-specific detection rates follow the same ordering as the pooled rates; provider-specific paired tests also find no statistically significant difference between obligation-guided and generic boundary tests.

Across the 200 Python-to-Java trials, bootstrap 95% intervals are 70.5% (64.0–76.5) for existing tests, 74.5% (68.5–80.5) for random/property tests, 81.0% (75.5–86.0) for the generic boundary-test baseline, and 79.0% (73.0–84.5) for obligation-guided validation. The McNemar comparison between the latter two methods has 14 discordant trials, split 9 vs. 5 ($p = 0.424$). The observed detection rates are close; the test-count and provenance measures describe other differences between the methods.

Table 3. Python-to-Java statistical evidence. McNemar tests are paired over $n = 200$ subject-provider trials. Provider rows use 100 subjects each. Asterisks mark comparisons significant under Bonferroni $p < 0.0083$.

View	Row / provider	Col. / mode	b or OpenAI	c or Gemini	p or Δ
McNemar	Existing	Random	0	8	0.0078*
McNemar	Existing	LLM	0	21	0.0000*
McNemar	Existing	Obligation	0	17	0.0000*
McNemar	Random	LLM	0	13	0.0002*
McNemar	Random	Obligation	0	9	0.0039*
McNemar	LLM	Obligation	9	5	0.4240
Provider detection	Existing	—	63.0%	78.0%	+15.0
Provider detection	Random	—	68.0%	81.0%	+13.0
Provider detection	LLM	—	76.0%	86.0%	+10.0
Provider detection	Obligation	—	74.0%	84.0%	+10.0
Provider detection	Repair	—	74.0%	84.0%	+10.0
Provider McNemar	OpenAI	Obl. vs. LLM	3	5	0.7266
Provider McNemar	Gemini	Obl. vs. LLM	2	4	0.6875
Efficiency	Wilcoxon	Obl. < LLM tests	$W = 5619$	$\Delta = -0.85$	2.0×10^{-6}

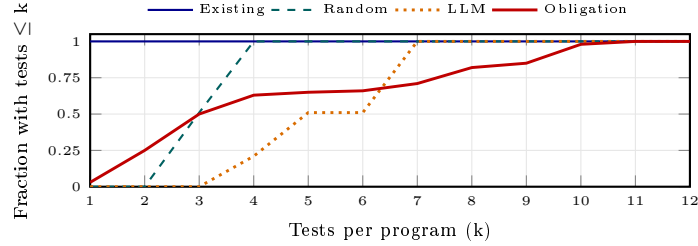


Fig. 4. Empirical CDF of Python-to-Java validation suite size. Obligation-guided suites reach the median by 4 tests and 90% of trials by 10. The figure shows suite-size distributions; execution time is not measured.

Packet availability matches guided detection because a packet is emitted for each guided mismatch. Table 1 reports this availability for both language pairs. The packets include the failing input, matched observations, provenance, and oracle metadata. The study does not compare repair outcomes with and without packet information.

Under the primary UB-safe rule, Rust behavior on C executions classified as undefined by UBSan is not counted as a mismatch. Under the policy-off rule, A-integer detection is 63.6% and overall C-to-Rust detection is 41.7%. D-string detection is 33.3% under both policies. These descriptive results depend on the selected equivalence policy.

5 Related Work

LLM code translation. Recent work studies class-level and repository-level translation, C-to-Rust migration, GPU transpilation, and pseudocode-mediated translation [23,13,2,11,6]. Several systems and benchmarks emphasize that memory safety, target-language idioms, and equivalence evidence remain central in real migration [15,19,18,28]. None makes source behavioral obligations the explicit validation unit. SageTransVal is complementary: it provides a translator-agnostic validation layer whose tests carry source-side justifications.

Validation and functional testing of translation. Compilation alone is insufficient for translation quality, and prior work has examined verified or validated transpilation [25,3], functional-property testing [8], and equivalence-bearing witnesses [10]. A recent survey maps LLM-agent integration and output validation in static analysis [12]. SageTransVal differs by associating function-level tests with source obligation families and by exporting structured packets for detected mismatches. The evaluation reports detection, suite size, coverage, and packet availability rather than repair effectiveness.

Compiler fuzzing and symbolic execution. Compiler fuzzing and hybrid fuzzing uncover deep semantic bugs that compilation misses [26,16,27,9]. SageTransVal

adapts the same idea to a black-box translator whose target code may be idiomatic but structurally distant from the source. The obligation taxonomy links source behavior to target observations when traditional intermediate representations are unavailable.

LLM debugging and repair. LLM-based repair systems consume execution feedback to localize and fix defects [4,14,22,24,21]. A quantum-program repair loop also executes candidate patches; its grounding results differ by defect type, helping API-migration cases but hurting semantic ones [17]. SageTransVal differs by producing structured counterexample packets and requiring full-suite revalidation after a candidate fix.

Generic boundary testing as a baseline. ChatUniTest-style generators produce boundary cases and assertions [7]. Our generic baseline is deterministic and template-based: it expands fixed value pools by type without invoking an LLM or external solver. It does not attach each test to a source obligation. The evaluation shows that raw detection is statistically close to obligation-guided validation, while guided validation remains more efficient and provides provenance.

6 Discussion and Related Work

On Python-to-Java, the generic baseline’s estimate is two points higher (81.0% versus 79.0%), but the paired difference is not significant. Guided validation uses 0.85 fewer tests per program and records family provenance; removing a family lowers coverage. Suite cost and provenance’s effect on decisions remain unmeasured.

For C-to-Rust, the primary UB-safe policy treats C executions flagged as undefined by UBSan as unconstrained. Under this policy, obligation-guided validation detects mismatches in 7 of 60 trials and the generic baseline in 6 of 60. With the policy disabled, overall detection is 41.7%, primarily due to A-integer cases; D-string detection remains 33.3%. These 60 trials are reported descriptively and do not support a broad ranking of methods.

Related work spans LLM translation and migration [2,19,28], verified transpilation [25,3], compiler fuzzing [26,9,27], and execution-guided repair [4,14]. SageTransVal complements these systems with source-justified tests, conservative observations, and revalidation-ready counterexamples rather than a new translator or repair model.

Prior translation studies examine behavioral preservation, memory safety, target idioms, repository validation, and functional-property testing [15,13,8,25,3]. SageTransVal instead evaluates function-level, obligation-associated tests and structured mismatch records; repository migration and downstream repair success remain outside scope.

The methods may be complementary: generic tests have the higher detection point estimate and provide broad inputs, while guided tests retain source-family provenance. Packets link mismatches to inputs and observations; downstream value and deployment effects remain untested.

Limitations. Mismatch detection is not semantic proof. The extractor reports branch/control-flow, boundary/type, API, and exception obligations; data flow, concurrency, floating point, ownership-sensitive C-to-Rust cases, and richer library semantics remain future work. Family coverage is not source-code coverage, and packet availability is not repair effectiveness.

Harness bugs and normalization choices can affect either language pair; we log compile, runtime, timeout, and observation outcomes separately. Deterministic decoding and cached translations mitigate provider variation, though prompts and newer models may change absolute rates. Subjects are function-level, and the category-focused C-to-Rust results are descriptive; statistical claims come from Python-to-Java.

HumanEval/MBPP-style functions underrepresent heap aliasing, callbacks, state, I/O, concurrency, and project builds; the C-to-Rust corpus omits ownership inference, lifetime-sensitive APIs, layout-sensitive structs, and unsafe-code encapsulation. Results concern selected function-level tasks, not repository migration.

Future work will extend obligations to data flow, richer oracle policies, and repository adapters; repair-agent studies can test whether packets and full-suite revalidation catch overfitted fixes.

7 Conclusion

We presented SageTransVal, which generates obligation-based tests, compares cross-language observations, and records mismatches in structured packets. On Python-to-Java, detection was 79.0% versus 81.0% for the generic baseline (not significantly different), with 0.85 fewer tests per program. On 60 C-to-Rust trials, rates were 11.7% and 10.0%, respectively; these results are descriptive. We measure family coverage and packet availability, not downstream repair effectiveness.

Acknowledgements This research is funded by Ho Chi Minh City University of Technology - VNU-HCM under grant **number T-KHMT-2026-04**.

References

1. Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C.: Program synthesis with large language models. arXiv (Cornell University) (2021). <https://doi.org/10.48550/arxiv.2108.07732>, <http://arxiv.org/abs/2108.07732>
2. Bedell, S., Siavash, N., Moin, A.: Llm4c2rust: Large language models for automated memory-safe code transpilation. arXiv (Cornell University) (2026), <https://arxiv.org/abs/2604.15485>
3. Bhatia, S., Qiu, J., Hasabnis, N., Seshia, S.A., Cheung, A.: Verified code transpilation with llms. In: Proceedings of the Conference on Neural Information Processing Systems (NeurIPS). Curran Associates (2024). <https://doi.org/10.48550/arxiv.2406.03003>, <http://arxiv.org/abs/2406.03003>

4. Bouzenia, I., Devanbu, P., Pradel, M.: Repairagent: An autonomous, llm-based agent for program repair. In: Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE). ACM/IEEE (2024). <https://doi.org/10.1109/ICSE55347.2024.00000>, <http://arxiv.org/abs/2403.17134>
5. Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G.: Evaluating large language models trained on code. arXiv preprint (2021). <https://doi.org/10.48550/arxiv.2107.03374>, <http://arxiv.org/abs/2107.03374>
6. Chen, S., Xu, C., Chen, J., Cao, J., Wu, J., Cheung, S.C.: Can emulating semantic translation help llms with code translation? a study based on pseudocode. ACM Transactions on Software Engineering and Methodology (2026). <https://doi.org/10.1145/3790101>, <http://arxiv.org/abs/2510.00920>
7. Chen, Y., Hu, Z., Chen, Z., Han, J., Deng, S., Yin, J.: Chatunitest: A framework for llm-based test generation. arXiv (Cornell University) (2023). <https://doi.org/10.48550/arxiv.2305.04764>, <http://arxiv.org/abs/2305.04764>
8. Eniser, H.F., Wustholz, V., Christakis, M.: Automatically testing functional properties of code translation models. arXiv (Cornell University) (2023). <https://doi.org/10.48550/arxiv.2309.12813>, <http://arxiv.org/abs/2309.12813>
9. Gerasimov, A.Y., Sargsyan, S., Kurmangaleev, S., Hakobyan, J., Asryan, S., Ermakov, M.: Combining dynamic symbolic execution, code static analysis and fuzzing. Proceedings of the Institute for System Programming of RAS (2018). [https://doi.org/10.15514/ispras-2018-30\(6\)-2](https://doi.org/10.15514/ispras-2018-30(6)-2), [https://doi.org/10.15514/ispras-2018-30\(6\)-2](https://doi.org/10.15514/ispras-2018-30(6)-2)
10. Heakl, A., Hashmi, S., Abi, C., Lee, C., Mahmoud, A.: Guaranteed guess: A language modeling approach for cisc-to-risc transpilation with testing guarantees. arXiv preprint (2025). <https://doi.org/10.48550/arxiv.2506.14606>, <http://arxiv.org/abs/2506.14606>
11. Heakl, A., Hashmi, S., Stahl, G.B., Han, S.H.E., Khan, S.A., Mahmoud, A.: Cass: Nvidia to amd transpilation with data, models, and benchmark. arXiv preprint (2025). <https://doi.org/10.48550/arxiv.2505.16968>, <https://arxiv.org/abs/2505.16968>
12. Hua, H.Q.B., Le, X.B.: Emerging trends and challenges toward llm agents in static analysis. In: Proceedings of IEA/AIE 2026. pp. 101–115. Lecture Notes in Computer Science (2026). https://doi.org/10.1007/978-981-92-2885-0_9
13. Ibrahimzada, A.R., Ke, K., Pawagi, M., Abid, M.S., Pan, R., Sinha, S., Jabbarvand, R.: Alphasprings: A neuro-symbolic compositional approach for repository-level code translation and validation. Proceedings of the ACM on software engineering. (2025). <https://doi.org/10.1145/3729379>, <http://arxiv.org/abs/2410.24117>
14. Kang, S., Chen, B., Yoo, S., Lou, J.G.: Explainable automated debugging via large language model-driven scientific debugging. Empirical Software Engineering (2024). <https://doi.org/10.1007/s10664-024-10594-x>, <https://doi.org/10.1007/s10664-024-10594-x>
15. Khatry, A., Zhang, R., Pan, J., Wang, Z., Chen, Q., Durrett, G., Dillig, I.: Crust-bench: A comprehensive benchmark for c-to-safe-rust transpilation. arXiv preprint (2025). <https://doi.org/10.48550/arxiv.2504.15254>, <http://arxiv.org/abs/2504.15254>
16. Kim, K., Jeong, D.R., Kim, C.H., Jang, Y., Shin, I., Lee, B.: Hfl: Hybrid fuzzing on the linux kernel. Proceedings of the Network and Distributed System Security Symposium (NDSS) (2020). <https://doi.org/10.14722/ndss.2020.24018>, <https://doi.org/10.14722/ndss.2020.24018>

17. Le, X.B., Tran-Truong, P.T., Nguyen, D.: When grounding hurts: Retrieval and validation for llm repair of quantum programs. In: Proceedings of ICECCS 2026 (2026), <https://lexuanbach.github.io/publication/ICECCS2026.pdf>, to appear
18. Li, S., Jiang, J., Ren, M., Zhong, S., TAN, H., Gou, Y., Han, X., Chong, C.Y., Peng, Y., Shen, J.: Codepivot: Bootstrapping multilingual transpilation in llms via reinforcement learning without parallel corpora. arXiv (Cornell University) (2026), <https://arxiv.org/abs/2604.18027>
19. Shiraishi, M., Cao, Y., Shinagawa, T.: Smartc2rust: Iterative, feedback-driven c-to-rust translation via large language models for safety and equivalence. arXiv preprint (2024). <https://doi.org/10.48550/arxiv.2409.10506>, <http://arxiv.org/abs/2409.10506>
20. Tran-Truong, P.T., Le, X.B.: Distributional program analysis: Treating large language models as program samplers. In: Proceedings of ASE 2026 (NIER Track) (2026). <https://doi.org/10.1145/3832783.3834555>, to appear
21. Wei, Y., Xia, C.S., Zhang, L.: Copiloting the copilots: Fusing large language models with completion engines for automated program repair. Proceedings of the International Conference on Automated Software Engineering (ASE) (2023). <https://doi.org/10.1145/3611643.3616271>, <http://arxiv.org/abs/2309.00608>
22. Xin, Q., Wu, H., Reiss, S.P., Xuan, J.: Towards practical and useful automated program repair for debugging. arXiv (Cornell University) (2024). <https://doi.org/10.48550/arxiv.2407.08958>, <http://arxiv.org/abs/2407.08958>
23. Xue, P., Wu, L., Yang, Z., Wang, C., Li, X., Zhang, Y., Li, J., Jin, R., Pei, Y., Shen, Z., Lyu, X., Keung, J.W.: Classeval-t: Evaluating large language models in class-level code translation. arXiv (Cornell University) (2024). <https://doi.org/10.48550/arxiv.2411.06145>, <http://arxiv.org/abs/2411.06145>
24. Yang, A.Z.H., Kolak, S., Hellendoorn, V.J., Martins, R., Goues, C.L.: Revisiting unnaturalness for automated program repair in the era of large language models. arXiv preprint (2024). <https://doi.org/10.48550/arxiv.2404.15236>, <http://arxiv.org/abs/2404.15236>
25. Yang, A.Z.H., Takashima, Y., Paulsen, B., Dodds, J., Kroening, D.: Vert: Verified equivalent rust transpilation with large language models as few-shot learners. In: Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE). ACM/IEEE (2024). <https://doi.org/10.48550/arxiv.2404.18852>, <http://arxiv.org/abs/2404.18852>
26. Yang, C., Deng, Y., Lu, R., Yao, J., Liu, J., Jabbarvand, R., Zhang, L.: Whitefox: White-box compiler fuzzing empowered by large language models. In: Proceedings of the ACM on Programming Languages (PACMPL). ACM (2024). <https://doi.org/10.1145/3689736>, <http://arxiv.org/abs/2310.15991>
27. Zhang, G., Shuai, Z., Ma, K., Liu, K., Chen, Z., Wang, J.: Fdse: Enhance symbolic execution by fuzzing-based pre-analysis (competition contribution). In: Proceedings of FASE 2024. Springer (2024). https://doi.org/10.1007/978-3-031-57259-3_16
28. Zhang, H., David, C.C., Wang, M., Paulsen, B., Kroening, D.: Scalable, validated code translation of entire projects using large language models. In: Proceedings of PLDI 2025. ACM (2025). <https://doi.org/10.1145/3729315>